

Supervised ML Study Guide

Chapter 1:

Supervised Learning is the main branch of Machine Learning. The *difference* between Supervised and Unsupervised Learning is that Supervised has a known Target or Label that we want to predict. The main types of Supervised Learning problems are regression (predicting a continuous variable) and classification (predicting the probability of belonging to a category)

When we train a machine learning model, we are fitting an algorithm on input data. The algorithm then identifies patterns between that input data and the known target. Once the model is finished "learning", it can be used for predictions, assuming that the found patterns really do exist in the wider world.

When *evaluating* a model, we want to understand its expected performance on that unseen, wider world data. To do this, we typically train our model on 70-80% of our data (the training set) and then evaluate the model on the withheld data (the testing set).

When a model identifies patterns in the training data that don't exist in the testing data, this is known as overfitting. You may also hear underfitting. This means the model has not fully understood all of the patterns in the training set and is typically used when describing models where training was cut short or interrupted.

We discussed two families of algorithms:

Linear Models

- [Linear models](#) data back well over 200 years.
- The basic idea is that the output can be expressed as a weighted sum of the input features plus a bias term ($y = b_1 \cdot x_1 + b_2 \cdot x_2 \dots + b_0$) or ($y = mx + b$)
- They are rigorously PROVEN in mathematics. Problems can only have one solution.
 - e.g The [Proof of Least Squares](#) for Linear Regression (OLS)

Pros of Linear Models

- They are very explainable. Every Linear Model can be written as an equation (no matter how complex)

- They are rooted in mathematical principles. There can only be one answer to the optimal equation.
- Not very computationally expensive. People were doing them with pencil/paper, so it's fairly trivial for a computer

Cons of Linear Models

- They typically require certain [mathematical assumptions](#) to be made
- Big ones are LINEARITY (i.e, correlations, some sort of linear relationship)
- and INDEPENDENCE (no multi-collinearity within the feature set)
- They are unable to capture nuanced relationships between complex features at scale

With the problem of multi-collinearity, we introduced two solutions:

- Principal Component Analysis (transforms the inputs into an independent feature space)
 - Dates back over 100 years and is a classical solution
 - Can also be used for dimensionality reduction if needed
 - Cons: Loses all explainability of the feature space
- [Regularization](#)
 - L1 Regularization: Send the weights of unimportant features to 0
 - L2 Regularization: Send the weights of unimportant features to a very small number approaching 0 but never equaling 0
 - Regularization helps avoid overfitting by de-prioritizing "noisy" or irrelevant features
 - This is a much more recent phenomenon, with its application only being developed in the last 25-50 years.

Fun fact: Regularization is super important and will make a cameo appearance later on when we mention boosted trees. It is also an important parameter when training neural networks.

Examples of Linear Models include:

Regression:

- Linear Regression (OLS)
- Ridge Regression (L2 Regularization)
- Lasso Regression (L1 Regularization)
- ElasticNet Regression (L1 + L2 Regularization)

Classification:

- Logistic Regression (automatically has some [regularization built in](#) with modern applications)

Chapter 2:

Tree Models

Tree-based models use [decision trees](#) to split the data into subsets based on the values of input features. The splits are made recursively to create a tree-like structure where each node represents a decision based on an input feature, and each leaf represents the output prediction.

Each split is based on an if-else statement. The value used for the split is "learned" using initial random values that update and converge as the model trains (note the similarity to KMeans, another random based model). The patterns and splits are very reliant on sample size, as each tree branch essentially narrows down to a particular set of rows which are then aggregated to generate a prediction.

For example, in our Pet Adoption example, let's say in our training data 10 people traveled for work & also traveled more than 1 week a month. Of those 10 people, 8 adopted Pet Rock. This means our model will predict a probability of 0.8 for that category if that was our input. Let's say that 20 people in our training set Did NOT travel for work & liked to dress up their pet. Of those 20 people, 12 adopted Dog, 6 adopted Cat, and 2 adopted Rock. If we provide that input to the model, we would get a 60% chance of Dog, 30% Cat, 10% Rock, so the final prediction would be Dog. Note that the model has to learn which features to use for splits and the values those features need to take.

As the splits get deeper and more granular, a larger sample size is needed. Typically, tree-based models need medium-large datasets to train effectively. With too few rows, tree-based models tend to be under or overfit, so a linear model might be a better approach.

[Gradient Boosting](#)

Tree Based models date back about 50 years in theory and 20 years in application, with many improvements in the 2010s.

Types of Trees:

[Docs](#)

Decision Trees: The simplest form of tree-based models. A decision tree is a flowchart-like structure where each internal node represents a "split" on an input feature, each branch represents the outcome of the split, and each leaf node represents the predicted output.

Random Forest: An ensemble method that builds multiple decision trees (hence "forest") and merges their results. The idea is to reduce overfitting and improve accuracy by averaging the predictions of many trees. The "random" in the name comes from the fact that each tree in the forest is trained on a random sub-sample of the provided training data. All trees are treated equally.

Gradient Boosting Machines (GBM): A powerful ensemble method that builds trees sequentially. Each new tree is trained to correct the errors made by the previous trees. Strong learners are prioritized (boosted) while weak learners are de-prioritized using regularization. This method includes popular implementations like XGBoost, LightGBM, and CatBoost.

Extra Trees (Extremely Randomized Trees): Similar to Random Forest, but with more randomness in the tree-building process. It randomly selects the feature and threshold for splitting, which can lead to more diverse trees and potentially better generalization.

AdaBoost (Adaptive Boosting): Another ensemble method that builds trees sequentially, but with a focus on boosting the performance of weak learners (often shallow decision trees called "stumps"). In AdaBoost, each tree is trained on the errors of the previous trees, and the model gives more weight to misclassified data points in subsequent rounds. The final prediction is made by combining the weighted predictions of all trees. AdaBoost is particularly effective for binary classification tasks and is known for improving the performance of weak classifiers, i.e. reduces overfitting.

Note that all of these Tree models can be used for both Regression and Classification problems.

The best tree-based models are all less than a decade old and include XGBoost, LightGBM, and CatBoost. These are all separate libraries from scikit-learn and are the current peak of non-neural network models.

Pros of Tree-based models:

- Can capture non-linear effects of input features
- Flexible and scale well
- Not limited by multi-collinearity or other mathematical assumptions
- Fairly explainable due to Feature Importances
- Typically outperforms linear models on larger datasets, especially in non-linear effects are present

Cons of Tree-based models:

- Very prone to overfitting, even boosted forests still overfit
- Require a decent sized data set to train on
- Have many options (hyperparameters) that can be tweaked. Typically increases complexity and time to build experiment
- Random based. A different random_state might provide a different model
- More complicated and harder to explain than linear models

Prof Note: Typically, I use LinearRegression, ElasticNet, RandomForestRegressor, AdaBoost, XGBoost/LightGBM as my models of choice in a manual regression experiment. The linear models are a baseline, with ElasticNet expected to be the best linear model. In a tie, you always choose the less complicated and more explainable model. The RandomForest is a baseline for the other trees. XGBoost/LightGBM are the typically expected best model. AdaBoost is a good backup option if the Boosted forests still overfit.

Chapter 3:

Regression Model Evaluation

When evaluating the performance of a regression model, we examine multiple metrics and visualizations. Not only do we want to quantify performance, but we also want to identify bias and limitations in the model.

We discussed 4 main metrics that are all based off of the [residuals](#) (the difference between the prediction and the observed target): R-Squared, Mean Squared Error, Root Mean Squared Error, Mean Absolute Error.

- R^2 is a statistical measure that represents the proportion of the variance in the dependent variable that is predictable from the independent variables.
- MSE measures the average squared difference between the actual and predicted values.
- RMSE provides a more interpretable measure of error than MSE, as it is in the same units as the dependent variable.
- MAE measures the average magnitude of the errors in the predictions, without considering their direction (positive or negative).

Limitations:

- R^2 : Can be misleading in non-linear models, doesn't penalize complexity, and doesn't necessarily reflect predictive power.
- MSE: Sensitive to outliers and less interpretable due to squared units.
- RMSE: Still sensitive to outliers, although more interpretable than MSE.
- MAE: Less sensitive to large errors, which might be a drawback in certain contexts where large errors are particularly problematic.

Choosing the Right Metric:

- Use R^2 when you need to understand the proportion of variance explained by the model.
- Use MSE when you want to penalize larger errors more strongly.
- Use RMSE when you need an interpretable metric in the same units as the output variable, but still want to penalize larger errors.
- Use MAE when you want a straightforward average error metric without emphasizing large errors.

In my opinion:

- Use R^2 in a vacuum. Is this one model somewhat decent?
- Use MSE & MAE in model comparison. Is this one model better than another model?
- MSE is typically the metric to optimize when using AutoML for regression problems
- Use RMSE & MAE to quantify uncertainty in your predictions as well as detect possible outliers
- If a model is better in MSE, MAE & R^2 than another model, then you should be very confident.
- TL;DR: Don't just use R^2 ...

You might also see Median Absolute Error or Median Squared Error thrown around too, but don't worry about them.

Visualizations:

We also discussed 2 visualizations for model evaluation: **Predicted vs Actual Plot & Residual Plot:**

From our friendly neighborhood ChatGPT:

Predicted vs. Actual Plot

Purpose:

The predicted vs. actual plot compares the predicted values from the model against the actual values from the dataset.

How to Interpret:

- Ideal Scenario: The points should lie close to the 45-degree diagonal line ($y = x$ line). This indicates that the predicted values are close to the actual values.
- Patterns to Watch For:
 - Deviation from the Line: Points scattered far from the line suggest poor predictions.
 - Systematic Deviations: A consistent pattern, such as all points lying above or below the line, indicates that the model might be biased or missing key relationships in the data.
 - Spread of Points: A wider spread of points around the line indicates higher variability in prediction errors, which may suggest that the model isn't capturing all the information in the data.

Benefits:

- Detecting Systematic Errors: Helps identify whether the model consistently underestimates or overestimates the target variable.
- Assessing Model Accuracy: Provides a visual summary of how well the model predictions align with actual outcomes.

Residual Plot

Purpose:

A residual plot shows the residuals (errors) on the y-axis and the predicted values (or sometimes actual values) on the x-axis. Residuals are the differences between the actual values and the predicted values.

How to Interpret:

- Ideal Scenario: Residuals should be randomly scattered around zero, without any discernible pattern. This suggests that the model's errors are random and not systematic.
- Patterns to Watch For:
 - Non-Random Patterns: A pattern (e.g., a curve or trend) in the residuals suggests that the model is missing some relationship in the data, indicating a poor fit.
 - Heteroscedasticity: If the residuals fan out (increase in spread) as the predicted values increase, it suggests heteroscedasticity, meaning that the model's variance is not constant across all levels of the independent variables.
 - Outliers: Points far from the zero line may indicate outliers or influential data points that the model doesn't handle well

Benefits:

- Diagnosing Model Fit: Helps to visually assess if the model is correctly capturing the underlying structure of the data.

- Checking for Homoscedasticity: Homoscedasticity (constant variance of errors) is an important assumption in regression models. The residual plot can help detect violations of this assumption.
- Identifying Non-Linearity: Non-linear patterns in the residuals suggest that the model may not be adequately capturing a non-linear relationship in the data.

Summary: Why Use Both?

- Predicted vs. Actual Plot: Offers a direct comparison between the predictions and the true values, giving an overall sense of the model's accuracy.
- Residual Plot: Provides insight into the errors the model makes, helping to diagnose issues like non-linearity, bias, and heteroscedasticity that might not be apparent from the predicted vs. actual plot.

TL:DR: The visualization plots allow us to identify bias in the model predictions. Are we bad at predicting low or high values? Is there a pattern, or are we systematically, making poor predictions? Are there outlier predictions or points that are skewing our MSE/MAE?

Individual METRICS provide a HOLISTIC interpretation of the model, while visualizations provide insight into GRANULAR patterns in model performance.

Chapter 4:

Typical Supervised Problem Workflow

- Data Preprocessing
 - Imputation (null values)
 - Encoding (categorical values -> numeric)
 - Scaling of Numeric Features (StandardScaler or MinMaxScaler)
 - Feature creation (do you want to create or transform columns? Like add them together, do binning, etc)
- Correlation Analysis
 - Are we afflicted by multi-collinearity?
 - Which features have high linear correlations with the target?

- NOTE: Just because a feature has a low correlation doesn't mean that it's useless. It could still have non-linear predictive power that a tree-based model can capture
- Feature Selection
 - Which columns will you use to train your model?
 - This is typically an iterative process
 - You might want to drop columns with high multi-collinearity or that have low correlations
 - On a first pass, might just want to keep everything and see how it goes #yolo
 - This could also be the step for PCA/Dimensionality Reduction if you so choose
- Create your Train/Test splits
 - i.e. 80/20, 75/25, 70/30

This is all data cleaning/engineering 🐱

- Begin your ML Experiment (an experiment is a collection of multiple algorithms and metrics used for tracking and reproducibility)
 - Train an assortment of algorithms (linear and tree-based)
 - Generate Training and Testing Metrics & Test Set Visualizations
 - Check for Overfitting for comparing the Training & Testing Metrics
 - Keep Track of the Test set metrics & visualizations for model comparisons
 - Compare all test set metrics and visualizations to find the "best" model
 - Use the visualizations to identify the model limitations
 - For the best model, try and look at Feature Importances
 - For features with low importance, it might be worth going back and dropping them in the Feature Selection step, then re-training the model
 - Fewer Columns = Less Complexity = More Explainability = More Computational Efficiency
 - Goal is Most Predictable with Least Complexity
- Tune best model hyperparameters as needed (optional)
- Retrain best model on ALL training data (not just the train/test split) (will do later)
- Save the model for future use in deployed production systems or ML applications (will do later)

This workflow is the same for both Regression & Classification. Classification models have different metrics & visualizations that we will review on Tuesday.

Chapter 5:

Vocabulary

- Supervised vs Unsupervised Machine Learning
- Features & Target/Label
- Scaling/Normalization
- Feature Engineering
- Data Preprocessing
- PCA
- Multi-collinearity/Independence
- Imputation
- Encoding
- Linear Models
- Ordinary Least Squares (OLS)
- Regression vs Classification
- Regularization (L1 & L2)
- Heteroscedasticity
- Decision Trees
- Ensemble models (Forest based models)
- Boosting (bonus: Bagging, Bootstrapping)
- R^2 , MSE, RMSE, MAE
- Predicted vs Actual Plot
- Residuals
- Model Evaluation
- Model Selection
- Feature Importances
- Train/Test split
- Model Explainability
- Model Experiment Tracking
- Bonus: Hyperparameter tuning, Cross-Validation, Random_State
- Bonus: MLOps, Model Deployment, MLFlow